

AI Test Automation Framework

Dossier de Preparation - Soutenance

Projet complet : Architecture, Auto-Healing, CI/CD

Questions & Reponses pour Soutenance orale et technique

Genere le 24/06/2026

Projet NoveoCare - Mariem El Houche-Ext

Table des Matieres

1. Presentation du Projet

- 1.1 Contexte et Objectifs
- 1.2 Architecture Globale (5 Modules)
- 1.3 Stack Technologique

2. Description Detaillee des Modules

- 2.1 AI Engine (Python FastAPI + Colab)
- 2.2 Automation Framework (Java 17 + Selenium + Cucumber)
- 2.3 Dashboard API (FastAPI + PostgreSQL)
- 2.4 Dashboard UI (React 18 + Vite)
- 2.5 CI/CD Pipeline (GitHub Actions)

3. Flux Cles

- 3.1 Flux de bout en bout (End-to-End)
- 3.2 Flux Auto-Healing (3 niveaux)
- 3.3 Flux Authentification JWT
- 3.4 Pipeline CI/CD

4. Auto-Healing Approfondi

- 4.1 Algorithme de Scoring (formule $S_{agregé}$)
- 4.2 Les 3 modes de guerison
- 4.3 Metriques et Dashboard

5. Q&A - Soutenance Presentation

- 5.1 Questions generales sur le projet
- 5.2 Questions sur l'architecture
- 5.3 Questions sur le self-healing
- 5.4 Questions sur l'authentification
- 5.5 Questions sur le pipeline CI/CD

6. Q&A - Soutenance Technique

- 6.1 Java / Maven / Selenium
- 6.2 Python / FastAPI / Authentification
- 6.3 React / Dashboard UI
- 6.4 Docker / CI-CD / GitHub Actions
- 6.5 Base de donnees PostgreSQL

7. Annexes

- 7.1 Configuration (config.properties)
- 7.2 Acces et URLs
- 7.3 Diagrammes PlantUML

1. Presentation du Projet

1.1 Contexte et Objectifs

Le projet AI Test Automation Framework a été développé pour NoveoCare, une application web de gestion de santé (backoffice). L'objectif est d'automatiser les tests fonctionnels de l'application de manière intelligente, en intégrant un mécanisme d'auto-réparation (self-healing) des localisateurs (locators) Selenium.

Les défis adressés par ce projet :

- Maintenance réduite : les localisateurs changent souvent avec les mises à jour UI, le self-healing permet d'éviter de réécrire les tests à chaque changement.
- Flaky tests : le pipeline CI/CD détecte et classe les tests flaky (re-exécution automatique des scénarios échoués).
- Visibilité : un dashboard centralisé avec des KPIs (taux de guérison, taux de réussite, historique) permet de monitorer la santé des tests.
- CI/CD intégré : GitHub Actions avec 13 suites de tests, exécution séquentielle sur self-hosted runner Docker.

1.2 Architecture Globale (5 Modules)

Le projet est divisé en 5 modules interconnectés :

AI Engine (Python FastAPI + Notebook Colab)

Moteur d'IA pour le self-healing. Expose un endpoint /heal qui analyse le DOM, calcule la similarité structurelle, sémantique et spatiale, et retourne un nouveau localisateur.

Automation Framework (Java 17 + Maven + Selenium + Cucumber)

Framework de tests fonctionnels. 13 suites de tests Cucumber, mécanisme de healing à 3 niveaux (cache Java -> baseline fichier -> API /heal), reporters vers le dashboard.

Dashboard API (FastAPI + PostgreSQL)

API REST pour stocker et exposer les résultats de tests et les métriques de healing. Authentification JWT (3 rôles) + API Key pour machine-to-machine.

Dashboard UI (React 18 + Vite)

Interface utilisateur pour visualiser les résultats des tests, les métriques de healing, lancer des exécutions et gérer les utilisateurs.

CI/CD (GitHub Actions + Docker)

Pipeline automatisé : build -> 13 suites de tests matricielles avec re-exécution -> classification flaky -> déploiement de la stack Docker (dashboard).

1.3 Stack Technologique

Module	Technologie	Version
Langage Backend	Python	3.13
Framework API	FastAPI	0.115+
Auth	PyJWT + passlib (bcrypt)	-
Base de données	PostgreSQL	16+
Langage Tests	Java	17
Build	Maven	3.9+
Test Framework	Cucumber	7.x
Browser Automation	Selenium	4.x
Frontend	React	18
Build Frontend	Vite	5.x
CI/CD	GitHub Actions	-
Conteneurisation	Docker + Docker Compose	-

2. Description Detaillee des Modules

2.1 AI Engine (Python FastAPI + Colab)

Le module AI Engine est le cerveau du self-healing. Il est compose de deux parties :

2.1.1 Dashboard API (dashboard_api.py)

Point d'entree de l'API FastAPI sur le port 8080. Expose les endpoints suivants :

- POST /auth/login - Authentification (email + password) -> retourne un JWT
- POST /api/users (CRUD) - Gestion des utilisateurs (admin only)
- POST /api/run-tests - Lancement d'une execution Maven avec tags Cucumber
- GET /api/dashboard - Resume des metriques de healing + resultats Cucumber
- POST /api/cucumber-runs - Push des resultats Cucumber (machine-to-machine)
- POST /api/healing-events - Push des evenements de healing
- POST /heal - Endpoint de healing (utilise par le framework Java)
- GET /api/metrics/history - Historique des metriques

2.1.2 Service d'authentification (auth_service.py)

Deux mecanismes d'authentification :

- JWT : HS256 avec expire de 480 minutes. 3 roles : admin, project_manager, qa_engineer.
- API Key : pour les communications machine-to-machine (poussee des resultats Cucumber).

Les mots de passe sont haches avec bcrypt via la librairie passlib. 3 comptes par default sont seedes au demarrage : administrateur, chef de projet, ingenieur QA.

2.1.3 Self-Healing Notebook (self_healing_v5.ipynb)

Notebook Colab qui implemente le pipeline complet de healing :

- Etape 1 - Baseline : lookup par MD5(type|valeur|hash du DOM) dans un cache fichier.
- Etape 2 - Extraction DOM : parsing HTML avec BeautifulSoup + lxml, filtrage des elements interactifs (input, button, a, select, textarea, label).
- Etape 3 - Similarite structurelle : tag (20%), attributs (50%), contexte DOM (30%).
- Etape 4 - Similarite semantique : SentenceTransformer paraphrase-multilingual-mpnet-base-v2.
- Etape 5 - Similarite spatiale : distance euclidienne normalisee entre les positions des elements.
- Score final : $S_{agregé} = 0.40 \times S_{structurelle} + 0.50 \times S_{semantique} + 0.10 \times S_{spatiale}$
- Seuil de succes : 0.75

2.2 Automation Framework (Java 17 + Selenium + Cucumber)

Framework de tests fonctionnels automatises pour l'application NoveoCare Backoffice. Architecture en couches :

2.2.1 Couche Page Objects (BasePage.java)

Classe parente de toutes les pages. La methode findElement(By) intercepte les NoSuchElementException et declenche le processus de healing :

- Capture de l'ecran (screenshot avec element encadre en rouge)
- Recuperation du HTML de la page
- Appel du HealingLocatorResolver pour trouver un nouveau localisateur
- Snapshot : sauvegarde du screenshot + payload JSON dans target/healing-debug/

2.2.2 Module Self-Healing (HealingLocatorResolver.java)

Resolution du healing en 3 niveaux (fallback) :

- Niveau 1 - Cache chaud (ConcurrentHashMap) : si le meme localisateur a deja ete gueri dans le meme run, on reuse le resultat.
- Niveau 2 - Baseline fichier (HealingBaseline.java) : fichier JSON persistant (target/healing-baseline.json) avec les guerisons des runs precedents.
- Niveau 3 - API /heal : appel HTTP POST vers l'API distante (Colab/ngrok ou CI Stub ou Dashboard API).

La classe SelfHealingClient.java est le client HTTP avec retry (2 tentatives). DashboardReporter.java pousse les evenements de healing vers l'API dashboard.

2.2.3 Runners et Organisation des Tests

GenericTagRunner.java : runner Cucumber parametre par tag, utilise pour 12 des 13 suites.

Les 13 suites de tests couvrent les fonctionnalites suivantes :

@login, @companies, @adherents, @structures, @appointments, @prescriptions, @billing, @documents, @users, @roles, @settings, @notifications, @logs

2.2.4 Configuration (config.properties)

Proprietes clees : self.healing.enabled=true, self.healing.api.url (URL du service de healing), self.healing.debug.request=true (sauvegarde des payloads de debug), dashboard.api.url=http://127.0.0.1:8080, dashboard.api.key (cle API).

2.3 Dashboard API (FastAPI + PostgreSQL)

API REST centralisee qui recoit et expose les donnees de test et de healing.

2.3.1 Base de donnees (3 tables principales)

Table cucumber_runs : stocke les resultats de chaque scenario Cucumber (feature_name, scenario, status, duration_ns, tags, run_id, classification).

Table healing_events : stocke chaque tentative de healing (old/new locator, scores, success, baseline_hit, healing_time_ms, elements counts).

Table metrics_snapshots : instantanes periodiques des metriques agregees depuis le Colab.

2.3.2 Resolution des metriques (fallback 3 niveaux)

La methode resolve_healing_payload() dans dashboard_service.py suit un fallback :

- 1. metrics_snapshots (table) : cherche un snapshot pour le run_id donne.
- 2. healing_events (table) : si pas de snapshot, agrege les evenements bruts (calcule healing_rate, baseline_hit_rate, avg_final_score, etc.).
- 3. SELF_HEALING_METRICS_URL (externe) : si pas de donnees en BDD, appelle le endpoint /metrics du Colab.

2.4 Dashboard UI (React 18 + Vite)

Interface utilisateur single-page (SPA) avec 4 pages principales :

- DashboardPage : vue d'ensemble avec KPIs (healing rate, baseline hit rate, avg score), graphiques d'evolution, tableau des resultats Cucumber.
- TestRunnerPage : lancement des tests (selection de tags), affichage des logs en temps reel (polling 2s).
- HealingPage : details de chaque evenement de healing (ancien/nouveau locator, scores, temps, statut) avec MiniChart pour chaque metrique.
- SettingsPage : configuration (URL API, run ID, dark mode, compte utilisateur).

2.5 CI/CD Pipeline (GitHub Actions)

Pipeline complet defini dans .github/workflows/ci-cd.yml.

Declencheurs : push sur main/develop (filtre paths: automation-framework/**, .github/workflows/**, pom.xml), pull_request,

workflow_dispatch (manuel avec choix de suite).

Jobs :

- build : compile le code (compile + test-compile) avec cache Maven (~/.m2/repository).
- feature-tests (matriciel) : 13 suites en sequence (max-parallel: 1) sur self-hosted runner. Chaque suite execute : health check du site -> tests primaires -> rerun des echoues -> classification flaky (TestClassifier) -> upload artifacts (rapports + debug).
- deploy : fusionne les artefacts, reconstruit et deploye la stack Docker (PostgreSQL + API + UI), pousse les resultats via DashboardPushMain.

3. Flux Cles

3.1 Flux de bout en bout (End-to-End)

1. User story (nouveau besoin) -> 2. FeatureGenerator (Colab IA) genere le fichier feature -> 3. Commit dans le repo -> 4. CI/CD declenche -> 5. Maven compile -> 6. Tests Cucumber executes (Selenium headless) -> 7. Auto-healing si locator casse -> 8. Rerun des echoues -> 9. TestClassifier (passe/flaky/failed) -> 10. Push des resultats au dashboard -> 11. Deploiement stack Docker.

3.2 Flux Auto-Healing (3 niveaux)

```
HealingWebDriver.findElement(By)
|
+-- NoSuchElementException
|
+-- [NIVEAU 1] Cache chaud (ConcurrentHashMap)
|   Si deja gueri dans ce meme run -> retour direct
|
+-- [NIVEAU 2] Baseline fichier (healing-baseline.json)
|   Si deja gueri dans un run precedent -> retour + baseline_hit=true
|
+-- [NIVEAU 3] API /heal (POST HTTP)
    Soit Colab (ngrok), soit CI Stub (port 8090), soit Dashboard API
    Reponse: {new_locator_type, new_locator_val, score, ...}
```

3.3 Flux Authentification JWT

```
1. Utilisateur -> POST /auth/login {email, password}
2. AuthService.find_user_by_email(email) -> SELECT * FROM app_users
3. AuthService.verify_password(plain, hash) -> bcrypt compare
4. AuthService.create_access_token(account) -> JWT (HS256, 480min)
   Payload: {sub: user_id, role: role, exp: expiry}
5. Reponse: {access_token, token_type: bearer, role, display_name, ...}
6. Toutes les routes protegees -> verify_token(headers.Authorization)
```

API Key : header X-API-Key pour les appels machine-to-machine (ex: push cucumber-runs depuis le CI/CD).

3.4 Pipeline CI/CD

```
Déclencheur (push/PR/manual)
|
+-- Job: build (compile + test-compile)
|
+-- Job: feature-tests (matrix: 13 suites)
|   Pour chaque suite:
|       1. Health check site staging
|       2. Tests primaires (xvfb-run mvn test)
|       3. Rerun des scenarios echoues
|       4. TestClassifier (passe/flaky/failed)
|       5. Upload artifacts (reports, debug)
|
+-- Job: deploy (branche main seulement)
    1. Download all artifacts
    2. docker compose down -v
    3. docker compose up -d --build
    4. DashboardPushMain pour chaque fichier JSON
```

4. Auto-Healing Approfondi

4.1 Algorithme de Scoring

Le score final est calcule selon la formule (version mise a jour avec le score spatial) :

$$S_{\text{agregé}} = 0.40 \times S_{\text{structurelle}} + 0.50 \times S_{\text{semantique}} + 0.10 \times S_{\text{spatiale}}$$

Seuil de succes : 0.75

S_structurelle (poids 0.40)

Composants :

- Tag HTML (20%) : correspondance exacte du type d'element.
- Attributs (50%) : placeholder, aria-label, title, name, data-testid, class, etc.
- Contexte DOM (30%) : parent immediat, elements voisins, profondeur dans l'arbre.

S_semantique (poids 0.50)

Utilise SentenceTransformer paraphrase-multilingual-mpnet-base-v2. Calcule la similarite cosinus entre les embeddings du texte de l'ancien element et du nouveau. Modele multilingue (français + anglais).

S_spatiale (poids 0.10)

Distance euclidienne normalisee entre les coordonnees (x, y, largeur, hauteur) des elements. Ajoutée récemment dans la version mise a jour du notebook pour ameliorer la precision du matching lorsque les coordonnees sont disponibles.

4.2 Les 3 modes de guerison

Mode 1 : Baseline Java (cache + fichier)

Usage : pendant l'execution des tests, pour eviter des appels reseau repetes.

Performance : aucun parsing DOM, aucun appel reseau. Cache memoire persistee dans target/healing-baseline.json.

Mode 2 : CI Healing Stub (Python standalone)

Usage : dans le pipeline CI/CD, quand le Colab n'est pas accessible.

Implementation : parse le DOM avec html.parser (stdlib, pas de ML). Matching par texte exact (1.0), texte partiel (0.8), attributs (0.7-0.9), traduction FR/EN (0.75). Score force a 0.95 en cas de succes.

Mode 3 : Moteur IA complet (Colab)

Usage : en production, avec acces GPU pour le modele NLP.

Pipeline complet : extraction DOM -> similarite structurelle -> similarite semantique (SentenceTransformer) -> similarite spatiale -> score agregé avec seuil 0.75.

4.3 Metriques et Dashboard

Metriques cles exposees dans le dashboard :

- Healing Rate : pourcentage de guerisons reussies (successful_healings / total_healing_requests)
- Baseline Hit Rate : pourcentage de guerisons resolues via le cache/baseline
- Avg Final Score : moyenne des scores finaux (S_agregé)
- Avg Healing Time : temps moyen de resolution du healing en millisecondes
- NLP Filter Efficiency : ratio elements envoyes au NLP / elements extraits
- Evolution graphique : historique des metriques sur les 50 derniers snapshots

5. Q&A - Soutenance Presentation

Cette section contient les questions possibles lors de la soutenance orale (presentation generale du projet) avec les reponses recommandees.

5.1 Questions generales sur le projet

Q: Quel est l'objectif principal de ce projet ?

R: Automatiser les tests fonctionnels du backoffice NoveoCare avec un mecanisme d'auto-reparation (self-healing) des localisateurs Selenium, afin de reduire la maintenance des tests et d'ameliorer la fiabilite du pipeline CI/CD.

Q: Quelle est la valeur ajoutee par rapport a un framework de test classique ?

R: Le self-healing permet d'eviter de reecrire les tests a chaque changement d'interface. Le pipeline detecte et classifie les tests flaky. Le dashboard offre une visibilite complete sur la sante des tests et l'efficacite du healing.

Q: Quels sont les defis techniques que vous avez rencontres ?

R: 1) Le matching des localisateurs dans un DOM complexe avec des elements dynamiques. 2) L'integration entre Java (framework de test) et Python (moteur IA). 3) Le timeout des sessions Colab/ngrok. 4) La classification flaky necessitant une re-execution fiable.

Q: Combien de temps a dure le developpement ?

R: Le projet a ete developpe sur plusieurs iterations. La phase initiale a couvert la creation du framework de test, suivie par l'ajout du self-healing (notebook Colab), puis l'integration du dashboard et du pipeline CI/CD.

5.2 Questions sur l'architecture

Q: Pourquoi avoir separe le moteur IA (Python) du framework de test (Java) ?

R: Python offre un ecosysteme NLP plus riche (SentenceTransformer, BeautifulSoup) et est plus adapte au prototypage rapide. Java est le standard pour les frameworks de test d'entreprise (Selenium + Cucumber + Maven). La separation permet aussi de deployer et faire evoluer chaque composant independamment.

Q: Comment les 5 modules communiquent-ils entre eux ?

R: Via des API REST. Le framework Java appelle le moteur IA via POST /heal. Le framework envoie les resultats au Dashboard via POST /api/cucumber-runs et POST /api/healing-events. Le Dashboard UI consulte l'API via GET. Le CI/CD orchestre le tout via GitHub Actions.

Q: Pourquoi avoir choisi PostgreSQL comme base de donnees ?

R: PostgreSQL est robuste, open-source, et bien adapte aux donnees structurees de ce projet. Il est facilement containerisable avec Docker et offre de bonnes performances pour les requetes analytiques (aggregation de metriques).

5.3 Questions sur le self-healing

Q: Comment fonctionne le self-healing ?

R: Quand Selenium ne trouve pas un element (NoSuchElementException), le framework capture le HTML de la page et appelle l'API de healing qui analyse le DOM, calcule la similarite entre l'ancien localisateur et chaque element candidat, et retourne le meilleur nouveau localisateur avec un score de confiance.

Q: Quels sont les 3 niveaux de healing et pourquoi ?

R: Cache chaud (meme run) pour la rapidite, baseline fichier (runs precedents) pour la persistance, API /heal pour les cas jamais rencontres. Ce fallback optimise le temps de reponse tout en garantissant une couverture maximale.

Q: Quelle est la formule du score et pourquoi ces poids ?

R: $S = 0.40 \times S_{\text{struct}} + 0.50 \times S_{\text{sem}} + 0.10 \times S_{\text{spat}}$. Le poids fort du semantique (50%) car le sens du texte est le meilleur indicateur pour retrouver un element. Le structurel (40%) capture les attributs techniques. Le spatial (10%) est un adjuvant qui utilise

les coordonnees quand elles sont disponibles.

Q: Quel est le taux de reussite du healing ?

R: Dans le CI Stub (sans ML), le taux est eleve grace au matching par attributs. Avec le Colab complet, le taux depend de la qualite du DOM mais le seuil de 0.75 garantit que seuls les matches fiables sont retournes.

5.4 Questions sur l'authentification

Q: Quels sont les deux mecanismes d'authentification ?

R: JWT (HS256, expire 480min) pour les utilisateurs humains avec 3 roles. API Key (via header X-API-Key) pour les communications machine-to-machine, notamment la pousse des resultats depuis le CI/CD.

Q: Quels sont les 3 roles et leurs permissions ?

R: admin : acces total (CRUD utilisateurs, lancement tests, toutes les metriques). project_manager : visualisation des resultats et metriques. qa_engineer : lancement des tests et visualisation.

5.5 Questions sur le pipeline CI/CD

Q: Pourquoi max-parallel: 1 dans la matrice de tests ?

R: Le self-hosted runner a des ressources limitees (CPU, memoire, navigateur). Executer les 13 suites en parallele provoquerait des conflits (ports, sessions navigateur) et des faux positifs. La sequence garantit des resultats fiables.

Q: Comment sont geres les tests flaky ?

R: Apres le premier run, les scenarios echoues sont re-executes automatiquement (rerun). Le TestClassifier compare les deux rapports JSON : passe aux deux = 'passed', passe au rerun seulement = 'flaky', echoue aux deux = 'failed'.

Q: Que fait le job 'deploy' exactement ?

R: Il telecharge tous les artefacts de test, arrete la stack Docker existante, la reconstruit et la relance, puis pousse tous les resultats Cucumber au dashboard via DashboardPushMain. Il ne s'execute que sur la branche main.

6. Q&A - Soutenance Technique

Cette section contient les questions techniques approfondies pour la soutenance technique.

6.1 Java / Maven / Selenium / Cucumber

Q: Comment est organise le projet Maven ?

R: Le projet a un pom.xml parent a la racine avec le module automation-framework. Le pom.xml d'automation-framework declare les dependances : Selenium 4.x, Cucumber 7.x (with JUnit Platform Suite), JUnit 5, Jackson (JSON), et Apache HttpClient pour les appels REST.

Q: Pourquoi utiliser JUnit Platform Suite (@Suite) pour Cucumber ?

R: Cela permet d'utiliser le nouveau moteur Cucumber avec JUnit 5, offrant une meilleure integration avec le reporting, les tags, et la re-execution. GenericTagRunner est configure avec @ConfigurationParameter pour les features, le glue code, et les plugins.

Q: Comment fonctionne HealingLocatorResolver en details ?

R: La methode resolve(By originalLocator, String pageSource) : 1) Verifie le cache chaud (ConcurrentHashMap<String, By>). 2) Si miss, verifie le fichier baseline (healing-baseline.json) charge au demarrage. 3) Si miss, construit un payload JSON (old locator, DOM, screenshots) et appelle SelfHealingClient qui fait POST /heal avec retry. 4) Met a jour le cache et le fichier baseline.

Q: Comment BasePage.findElement intercepte l'exception ?

R: La methode custom findElement(By) est la seule utilisee dans tous les Page Objects. Elle wrappe l'appel Selenium standard et capture NoSuchElementException. En cas d'exception, elle prend un screenshot (avec encadrement rouge de l'emplacement attendu), recupere le page source, et appelle le resolver.

Q: Quel est le role de DashboardReporter ?

R: C'est un hook Cucumber (@After) qui collecte les resultats de chaque scenario et les envoie au Dashboard API apres l'execution. Il pousse aussi les evenements de healing cumules pendant le run. Il implemente l'interface ConcurrentHashMap pour accumuler les evenements de facon thread-safe.

Q: Comment fonctionne le rerun des tests echoues ?

R: Cucumber genere un fichier failed-{suite}.txt via le plugin rerun. Dans le pipeline CI/CD, si ce fichier existe et n'est pas vide, Maven est relance avec -Dcucumber.features=@failed-{suite}.txt, ce qui execute uniquement les scenarios listes dans ce fichier.

6.2 Python / FastAPI / Authentification

Q: Comment FastAPI gere-t-il l'authentification ?

R: La fonction verify_token() est un middleware (FastAPI Depends) qui extrait le header Authorization: Bearer <token>, decode le JWT avec PyJWT, verifie la signature HS256 et l'expiration, puis injecte le payload (user_id, role) dans la requete. Les routes sont protegees par des dependences qui verifient le role.

Q: Comment le hash bcrypt est-il implemente ?

R: La librairie passlib avec le backend bcrypt genere un hash different a chaque appel (sel sale). Le hash est stocke en base et verifie via passlib.verify(plain_password, hash). 3 comptes par default sont crees au demarrage avec des mots de passe haches.

Q: Comment l'API Key est-elle validee ?

R: La fonction verify_api_key() lit le header X-API-Key et le compare a la variable d'environnement DASHBOARD_API_KEY. Si les deux correspondent, l'accès est autorise sans JWT (pour les communications machine-to-machine).

Q: Comment le lancement de tests Maven est-il integre dans l'API ?

R: La classe TestRunManager execute mvn test en subprocess avec les options configurees (tags, runner, plugins, healing enabled). Les logs sont streames en temps reel vers le dashboard via des endpoints websocket ou des fichiers de log polles toutes les 2 secondes.

6.3 React / Dashboard UI

Q: Comment les donnees sont-elles chargees dans le dashboard ?

R: Chaque page fait des fetch() vers l'API FastAPI avec le token JWT dans le header Authorization. HealingPage utilise Promise.all() pour charger les metriques, les evenements et l'historique en parallele.

Q: Comment est implemente le MiniChart ?

R: Composant SVG custom qui dessine une ligne (path) a partir d'un tableau de valeurs. Utilise un gradient lineaire pour le remplissage. Il gere les cas particuliers : 0 valeur (affiche 'No data'), 1 valeur (affiche un point avec '1 data point').

Q: Quelles sont les optimisations de performance ?

R: Le polling est limite a 2 secondes, la pagination des evenements (12 par page), le chargement parallele des donnees, et les appels API avec useCallback/useEffect propres (sans fuites memoires).

6.4 Docker / CI-CD / GitHub Actions

Q: Comment est configure le self-hosted runner ?

R: Le runner est Dockerise (Dockerfile dans tools/runner/) avec Chrome, chromedriver, xvfb (serveur X virtuel) et le logiciel GitHub Actions runner. Il est lance avec docker-compose et s'enregistre aupres de GitHub.

Q: Pourquoi utiliser xvfb-run ?

R: xvfb-run lance un serveur X virtuel (framebuffer) pour permettre a Chrome de fonctionner en mode headless sur un serveur sans ecran. --auto-servernum choisit un port de maniere dynamique pour eviter les conflits.

Q: Comment les secrets sont-ils geres ?

R: Les mots de passe (BACKOFFICE_USER_EMAIL/PASSWORD, DASHBOARD_API_KEY) sont stockes dans les secrets GitHub et injectes via \${ secrets.X }. Ils ne sont jamais visibles dans les logs. Le .env de la stack Docker contient la configuration sensible.

Q: Que manque-t-il pour un vrai CD (Continuous Deployment) ?

R: Le deploiement de l'application cible (NoveoCare backoffice) vers le staging ou la production. Actuellement, seul le dashboard de monitoring est deploye. Il faudrait ajouter des smoke tests post-deploiement et un rollback automatique.

6.5 Base de donnees PostgreSQL

Q: Quelle est la structure de la table healing_events ?

R: id (SERIAL PK), created_at (TIMESTAMP), scenario_name, old_locator_type, old_locator_val, new_locator_type, new_locator_val, success (BOOLEAN), score (FLOAT), structural_score, semantic_score, healing_time_ms (INT), baseline_hit (BOOLEAN), exception_type, elements_extracted, after_struct_filter, after_spatial_filter, sent_to_nlp, run_id (VARCHAR).

Q: Comment les donnees sont-elles inserees ?

R: Les evenements de healing sont inseres un par un via POST /api/healing-events depuis DashboardReporter (Java) ou depuis le Colab (Python). Les resultats Cucumber sont inseres en lot via POST /api/cucumber-runs. Les snapshots de metriques via POST /api/metrics.

Q: Comment la base est-elle initialisee au demarrage ?

R: Un script SQL (init.sql) dans le docker-compose cree les 3 tables et insere les donnees initiales. L'application FastAPI utilise un event de demarrage (@app.on_event('startup')) pour verifier la connexion et seeder les comptes utilisateurs si la table est vide.

7. Annexes

7.1 Configuration (config.properties)

```
# Fichier: automation-framework/src/main/resources/config.properties

# Backoffice
backoffice.url=https://stg-bo.noveocare.com/login
backoffice.user.email=
backoffice.user.password=

# Self-Healing
self.healing.enabled=true
self.healing.api.url=
self.healing.debug.request=true
self.healing.healed.by.cache=0
self.healing.healed.by.healing=0

# Dashboard API
dashboard.api.url=http://127.0.0.1:8080
dashboard.api.key=change-me
```

7.2 Accés et URLs

Composant	URL / Accès	Port
Dashboard API (FastAPI)	http://localhost:8080	8080
Dashboard UI (React)	http://localhost:5173 (dev) / 80 (prod)	5173 / 80
PostgreSQL	localhost:5432	5432
CI Healing Stub	http://localhost:8090	8090
Backoffice Staging	https://stg-bo.noveocare.com/login	443
Self-Hosted Runner	Docker container (tools/runner/)	-
Colab (via ngrok)	URL dynamique (ngrok)	-

Comptes par défaut (seed)

Role	Email	Mot de passe
Administrateur	admin@noveocare.com	(hache bcrypt)
Project Manager	pm@noveocare.com	(hache bcrypt)
QA Engineer	qa@noveocare.com	(hache bcrypt)

7.3 Diagrammes PlantUML

Les diagrammes de sequence et d'activité sont disponibles au format PlantUML dans le repertoire docs/diagrams/ :

- seq_authentication.puml - Authentification JWT
- seq_create_user.puml - Creation d'utilisateur
- seq_classification_reexecution.puml - Classification et re-execution
- seq_extraction_exposition_donnees.puml - Extraction et exposition des donnees
- activity_pipeline_ci_cd.puml - Pipeline CI/CD
- seq_create_category.puml - Creation de categorie (vide)